3 A Closer Look at Classes

Chapter Objectives

3.1 Assigning objects

3.2 Passing objects to functions

3.3 Returning objects from functions

3.4 An Introduction to friend functions

In this chapter you continue to explore the class. You will learn about assigning objects, passing objects to functions, and returning objects from functions. You will also learn about an important new type of function: the friend.

Review Skills Check

Before proceeding, you should be able to correctly answer the following questions and do the exercises.

1. Given the following class, what are the names of its constructor and destructor functions?

```
class widgit
{
    int x, y;
public:
    // ... fill in constructor and destructor functions
};
```

2. When is a constructor function called? When is a destructor function called?

3. Given the following base class, show how it can be inherited by a derived class called **Mars**.

```
class planet
{
    int moons;
    double dis_from_sun;
    double diameter;
    double mass;
public:
    // ...
};
```

4. There are two ways to cause a function to be expanded in line. What are they?

5. Give two possible restrictions to in-line functions.

6. Given the following class, show how an object called **ob** that passes the value 100 to **a** and X to **c** would be declared.

```
class sample
{
    int a;
    char c;
public:
    sample(int x, char ch) { a = x; c = ch; }
    // ...
};
```

3.1 ASSIGNING OBJECTS

One object can be assigned to another provided that both objects are of the same type. By default, when one object is assigned to another, a bitwise copy of all the data members is made. For example, the contents of object called **o1** is assigned to another object called **o2**, the

contents of all of **o1**'s data are copied into the equivalent members of **o2**. This is illustrated by the following program:

```
// An example of object assignment.
#include <iostream>
using namespace std;

class myclass
{
    int a, b;
public:
    void set(int i, int j) { a = i; b = j; }
    void show() { cout << a << ' ' << b << "\n"; }
};

int main()
{
    myclass o1, o2;

    o1.set(10, 4);

    // assign o1 to o2
    o2 = o1;

    o1.show();
    o2.show();

    return 0;
}
```

Here, object **o1** has its member variables **a** and **b** set to the values 10 and 4, respectively. Next, **o1** is assigned to **o2**. This causes the current value of **o1.a** to be assigned to **o2.a** and **o1.b** to be assigned to **o2.b**. Thus, when run, this program displays

```
10 4
10 4
```

Keep in mind that an assignment between two objects simply makes the data in those objects identical. The two objects are still completely separate, For example, after the assignment, calling **o1.set()** to set the value of **o1.a** has no effect on **o2** or its **a** value.

EXAMPLES

1. Only objects of the same type can be used in an assignment statement. If the objects are not of the same type, a compile-time error is reported. Further, it is not sufficient that the types just be physically similar-their type names must be the same. For example, this is not a valid program:

```
// This program has an error.
#include <iostream>
using namespace std;

class myclass
{
    int a, b;
```

```
public:
    void set(int i, int j) { a = i; b = j; }
    void show() { cout << a << ' ' << b << "\n"; }
};

/*
   This class is similar to myclass but uses a
   different class name and thus appears as a different
   type to the compiler.
*/
class yourclass
{
    int a, b;
public:
    void set(int i, int j) { a = i; b = j; }
    void show() { cout << a << ' ' << b << "\n"; }
};

int main()
{
    myclass o1;
    yourclass o2;

    o1.set(10, 4);

    o2 = o1; // ERROR, objects not of same type

    o1.show();
    o2.show();

    return 0;
}
```

In this case, even though **myclass** and **yourclass** are physically the same, because they have different type names, they are treated as differing types by the compiler.

2. It is important to understand that all data members of one object are assigned to another when an assignment is performed. This includes compound data such as arrays. For example, in the following version of the **stack** example, only **s1** has any characters actually pushed onto it. However, because of the assignment, **s2**'s **stck** array will also contain the characters **a**, **b**, and **c**.

```
#include <iostream>
using namespace std;

#define SIZE 10

// Declare a stack class for characters.
class stack
{
    char stck[SIZE]; // holds the stack
    int tos; // index of top of stack
public:
```

```
    stack(); // push character on stack
    char pop(); // pop character from stack
};

// Initialize the stack.
stack::stack()
{
    cout << "Constructing a stack\n";
    tos = 0;
}

// Push a character.
void stack::push(char ch)
{
    if(tos==SIZE)
    {
        cout << "Stack is full\n";
        return;
    }
    stck[tos] = ch;
    tos++;
}

// Pop a character.
char stack::pop()
{
    if(tos==0)
    {
        cout << "Stack is empty\n";
        return 0; // return null on empty stack
    }
    tos--;
    return stck[tos];
}

int main()
{   // Create two stacks that are automatically initialized.
    stack s1, s2;
    int i;

    s1.push('a');
    s1.push('b');
    s1.push('c');

    // clone s1
    s2 = s1; // now s1 and s2 are identical

    for(i=0; i<3; i++)
        cout << "Pop s1: " << s1.pop() << "\n";

    for(i=0; i<3;i++)
```

```
        cout << "Pop s2: " << s2.pop() << "\n";

    return 0;
}
```

3. You must exercise some care when assigning one object to another. For example, here is the **strtype** class developed in Chapter 2, along with a short **main()**. See if you can find an error in this program.

```
#include <iostream>
#include <cstring>
#include <cstdlib>
using namespace std;

class strtype
{
    char *p;
    int len;
public:
    strtype(char *ptr);
    ~strtype();
    void show();
};

strtype::strtype(char *ptr)
{
    len = strlen(ptr);
    p = (char *) malloc(len+1);
    if(!p)
    {
        cout << "Allocation error\n";
        exit(1);
    }
    strcpy(p, ptr);
}

strtype::~strtype()
{
   cout << "Freeing p\n";
   free(p);
}

void strtype::show()
{
    cout << p << " - length: " << len;
    cout << "\n";
}

int main()
{
    strtype s1("This is a test."), s2("I like C++.");
```

```
    s1.show();
    s2.show();

    // assign s1 to s2 - - this generates an error
    s2 = s1;

    s1.show();
    s2.show();

    return 0;
}
```

The trouble with this program is quite insidious. When **s1** and **s2** are created, both allocate memory to hold their respective strings. A pointer to each object's allocated memory is stored in **p**. When a **strtype** object is destroyed, this memory is released. However, when **s1** is assigned to **s2**, **s2**'s **p** now points to the same memory as **s1**'s **p**. Thus, when these objects are destroyed, the memory pointed to by **s2**'s **p** is freed *twice* and the memory originally pointed to by **s2**'s **p** is not freed *at all*.

While benign in this context, this sort of problem occurring in a real program will cause the dynamic allocation system to fail, and possibly even cause a program crash. As you can see from the preceding example, when assigning one object to another, you must make sure you are not destroying information that may be needed later.

EXERCISES

1. What is wrong with the following program?

```
// This program has an error.
#include <iostream>
using namespace std;

class cl1
{
    int i, j;
public:
    cl1(int a, int b) { i = a; j = b; }
    // ...
};

class cl2
{
    int i, j;
public:
    cl2(int a, int b) { i = a; j = b; }
    // ...
};

int main()
{
    cl1 x(10, 20);
    cl2 y(0, 0);
    x = y;
```

```
    // ...
}
```

2. Using the **queue** class that you created for Chapter 2, Section 2.1, Exercise 1, show how one queue can be assigned to another.

3. If the **queue** class from the preceding question dynamically allocates memory to hold the queue, why, in this situation, can one queue not be assigned to another?

3.2 PASSING OBJECTS TO FUNCTIONS

Objects can be passed to functions as arguments in just the same way that other types of data are passed. Simply declare the function's parameter as a class type and then use an object of that class as an argument when calling the function. As with other types of data, by default all objects are passed by value to a function.

EXAMPLES

1. Here is a short example that passes an object to a function:

```
#include <iostream>
using namespace std;

class samp
{
    int i;
public:
    samp(int n) { i = n; }
    int get_i() { return i; }
};

// Return square of o.i.
int sqr_it(samp o)
{
    return o.get_i() * o.get_i();
}

int main()
{
    samp a(10), b(2);

    cout << sqr_it(a) << "\n";
    cout << sqr_it(b) << "\n";

    return 0;
}
```

This program creates a class called **samp** that contains one integer variable called **i**. The function **sqr_it()** takes an argument of type **samp** and returns the square of that object's **i** value. The output from this program is 100 followed by 4.

2. As stated, the default method of parameter passing in C++, including objects, is by value. This means that a bitwise copy of the argument is made and it is this copy that is used by the function. Therefore, changes to the object inside the function do not affect the calling object. This is illustrated by the following example:

```
/*
   Remember, objects, like other parameters, are passed
   by value. Thus changes to the parameter inside a
   function have no effect on the object used in the call.
*/
#include <iostream>
using namespace std;

class samp
{
    int i;
public:
    samp(int n) { i = n; }
    void set_i(int n) { i = n; }
    int get_i() { return i; }
};

/*
   Set o.i to its square. This has no effect on the
   object used to call sqr_it(), however.
*/
void sqr_it(samp o)
{
    o.set_i(o.get_i() * o.get_i());

    cout << "Copy of a has i value of " << o.get_i();
}

int main()
{
    samp a(10);

    sqr_it(a); // a passed by value

    cout << "But, a.i is unchanged in main: ";
    cout << a.get_i(); // displays 10

    return 0;
}
```

The output displayed by this program is:

```
Copy of a has i value of 100
```

```
But, a.i is unchanged in main:  10
```

3. As with other types of variables, the address of an object can be passed to a function so that the argument used in the call can be modified by the function. For example, the

following version of the program in the preceding example does, indeed, modify the value of the object whose address is used in the call to **sqr_it()**.

```
/*
   Now that the address of an object is passed to sqr_it(),
   the function can modify the value of the argument whose
      address is used in the call.
*/
#include <iostream>
using namespace std;

class samp
{
    int i;
public:
    samp(int n) { i = n; }
    void set_i(int n) { i = n; }
    int get_i() { return i; }
};

/*
   Set o.i to its square. This affects the calling
   argument.
*/
void sqr_it(samp *o)
{
    o->set_i(o->get_i() * o->get_i());

    cout << "Copy of a has i value of " << o->get_i();
    cout << "\n";
}

int main()
{
    samp a(10);
    sqr_it(&a); // pass a's address of sqr_it()

    cout << "Now, a in main() has been changed: ";
    cout << a.get_i(); // displays 100

    return 0;
}
```

This program now displays the following output:

```
Copy of a has i value of 100
```

```
Now, a.i in main() has been changed:  100
```

4. When a copy of an object is made when being passed to a function, it means that a new object comes into existence. Also, when the function that the object was passed to terminates, the copy of the argument is destroyed. This raises two questions. First, is the object's constructor called when the copy is made? Second, is the object's destructor called when the copy is destroyed? The answer may, at first, seem surprising.

When a copy of an object is made to be used in a function call, the constructor function is *not* called. The reason for this is simple to understand if you think about it. Since a constructor function is generally used to initialize some aspect of an object, it must not be called when making a copy of an already existing object passed to a function. Doing so would alter the contents of the object. When passing an object to a function, you want the current state of the object, not its initial state.

However, when the function terminates and the copy is destroyed, the destructor function *is* called. This is because the object might perform some operation that must be undone when it goes out of scope. For example, the copy may allocate memory that must be released.

To summarize, when a copy of an object is created because it is used as an argument to a function, the constructor function is not called. However, when the copy is destroyed (usually by going out of scope when the function returns), the destructor function is called.

The following program illustrates the preceding discussion:

```
#include <iostream>
using namespace std;

class samp
{
    int i;
public:
    samp(int n)
    {
        i = n;
        cout << "Constructing\n";
    }
    ~samp() { cout << "Destructing\n"; }
    int get_i() { return i; }
};

// Return square of o.i.
int sqr_it(samp o)
{
    return o.get_i() * o.get_i();
}

int main()
{
    samp a(10);

    cout << sqr_it(a) << "\n";

    return 0;
}
```

This function displays the following:

```
Constructing
Destructing
100
```

Destructing

As you can see, only one call to the constructor function is made. This occurs when **a** is created. However, two calls to the destructor are made. One is for the copy created when **a** is passed to **sqr_it()**. The other is for **a** itself.

The fact that the destructor for the object that is the copy of the argument is executed when the function terminates can be a source of problems. For example, if the object used as the argument allocates dynamic memory and frees that memory when destroyed, its copy will free the same memory when its destructor is called. This will leave the original object damaged and effectively useless. (See Exercise 2, just ahead in this section, for an example.) It is important to guard against this type of error and to make sure that the destructor function of the copy of an object used in an argument does not cause side effects that alter the original argument.

As you might guess, one way around the problem of a parameter's destructor function destroying data needed by the calling argument is to pass the address of the object and not the object itself. When an address is passed, no new object is created, and therefore, no destructor is called when the function returns. (As you will see in the next chapter, C++ provides a variation on this theme that offers a very elegant alternative.) However, an even better solution exists, which you can use after you have learned about a special type of constructor called a *copy constructor*. A copy constructor lets you define precisely how copies of objects are made. (Copy constructors are discussed in Chapter 5.)

EXERCISES

1. Using the **stack** example from Section 3.1, Example 2, add a function called **showstack()** that is passed an object of type **stack**. Have this function display the contents of a stack.

2. As you know, when an object is passed to a function, a copy of that object is made. Further, when that function returns, the copy's destructor function is called. Keeping this in mind, what is wrong with the following program?

```
// This program contains an error.
#include <iostream>
#include <cstdlib>
using namespace std;

class dyna
{
    int *p;
public:
    dyna(int i);
    ~dyna() { free(p); cout << "freeing \n"; }
    int get() { return *p; }
};

dyna::dyna(int i)
{
    p = (int *) malloc(sizeof(int));
    if(!p)
    {
        cout << "Allocation failure\n";
        exit(1);
```

```
    }

    *p = i;
}

// Return negative value of *ob.p
int neg(dyna ob)
{
    return -ob.get();
}

int main()
{
    dyna o(-10);

    cout << o.get() << "\n";
    cout << neg(o) << "\n";

    dyna o2(20);
    cout << o2.get() << "\n";
    cout << neg(o2) << "\n";

    cout << o.get() << "\n";
    cout << neg(o) << "\n";

    return 0;
}
```

3.3 RETURNING OBJECTS FROM FUNCTIONS

Just as you can pass objects to functions, functions can return objects. To do so, first declare the function as returning a class type. Second, return an object of that type using the normal **return** statement.

There is one important point to understand about returning objects from functions, however: When an object is returned by a function, a temporary object is automatically created which holds the return value. It is this object that is actually returned by the function. After the value has been returned, this object is destroyed. The destruction of this temporary object might cause unexpected side effects in some situations, as is illustrated in Example 2 below.

EXAMPLES

1. Here is an example of a function that returns an object:

```
// Returning an object
#include <iostream>
#include <cstring>
using namespace std;

class samp
{
    char s[80];
```

```
public:
    void show() { cout << s << "\n"; }
    void set(char *str) { strcpy(s, str); }
};

// Return an object of type samp
samp input()
{
    char s[80];
    samp str;

    cout << "Enter a string: ";
    cin >> s;

    str.set(s);

    return str;
}

int main()
{
    samp ob;

    // assign returned object to ob
    ob = input();
    ob.show();

    return 0;
}
```

In this example, **input()** creates a local object called **str** and then reads a string from the keyboard. This string is copied into **str.s** and then **str** is returned by the function. This object is then assigned to **ob** inside **main()** when it is returned by the call to **input()**.

2. You must be careful about returning objects from functions if those objects contain destructor functions because the returned object goes out of scope as soon as the value is returned to the calling routine. For example, if the object returned by the function has a destructor that frees dynamically allocated memory, that memory will be freed even though the object that is assigned the return value is still using it. For example, consider this incorrect version of the preceding program:

```
// An error generated by returning an object.
#include <iostream>
#include <cstring>
#include <cstdlib>
using namespace std;

class samp
{
    char *s;
public:
    samp() { s = '\0'; }
    ~samp() { if(s) free(s); cout << "Freeing s\n"; }
```

```
    void show() { cout << s << "\n"; }
    void set(char *str);
};

// Load a string.
void samp::set(char *str)
{
    s = (char *) malloc(strlen(str)+1);
    if(!s)
    {
        cout << "Allocation error\n";
        exit(1);
    }

    strcpy(s, str);
}

// Return an object of type samp
samp input()
{
    char s[80];
    samp str;

    cout << "Enter a string: ";
    cin >> s;

    str.set(s);
    return str;
}

int main()
{
    samp ob;

    // assign returned object to ob
    ob = input(); // This causes an error!!!!
    ob.show();

    return 0;
}
```

The output from this program is shown here:

```
Enter a string:  Hello
Freeing s
Freeing s
Hello
Freeing s
Null pointer assignment
```

Notice that **samp**'s destructor function is called three times. First, it is called when the local object **str** goes out of scope when **input()** returns. The second time ~**samp()** is called is when the temporary object returned by **input()** is destroyed. Remember, when a n object is returned from a function, an invisible (to you) temporary object is automatically generated which holds the return value. In this case, this object is simply a copy of **str**, which is the return value of the function. Therefore, destructor is executed. Finally, the destructor for object **ob**, inside **main()**, is called when the program terminates.

The trouble is that in this situation, the first time the destructor executes, the memory allocated to hold the string input by **input()** is freed. Thus, not only do the other two calls to **samp**'s destructor try to free an already released piece of dynamic memory, but they destroy the dynamic allocation system in the process, as evidenced by the run-time message "Null pointer assignment." (Depending upon your compiler, the memory model used for compilation, and the like, you may or may not see this message if you try this program.)

The key point to be understood from this example is that when an object is returned from a function, the temporary object used to effect the return will have its destructor function called. Thus, you should avoid returning objects in which this situation is harmful. (As you will learn in Chapter 5, it is possible to use a copy constructor to manage this situation.)

EXERCISES

1. To illustrate exactly when an object is constructed and destructed when returned from a function, create a class called **who**. Have **who**'s constructor take one character argument that will be used to identify an object. Have the constructor display a message similar to this when constructing an object:

   ```
   Constructing who #x
   ```

   where **x** is the identifying character associated with each object. When an object is destroyed, have a message similar to this displayed:

   ```
   Destroying who #x
   ```

   where, again, **x** is the identifying character. Finally, create a function called **make_who()** that returns a **who** object. Give each object a unique name. Note the output displayed by the program.

2. Other than the incorrect freeing of dynamically allocated memory, think of a situation in which it would be improper to return an object from a function.

3.4 AN INTRODUCTION TO FRIEND FUNCTIONS

There will be times when you want a function to have access to the private members of a class without that function actually being a member of that class. Towards this end, C++ supports friend functions. A friend is not a member of a class but still has access to its private elements. Two reasons that friend functions are useful have to do with operator overloading and the creation of certain types of I/O functions. You will have to wait until later to see these uses of a friend in action. However, a third reason for friend functions is that there will be times when you want one function to have access to the private members of *two or more* different classes. It is this use that is examined here.

A friend function is defined as a regular, nonmember function. However, inside the class declaration for which it will be a friend, its prototype is also included, prefaced by the keyword **friend**. To understand how this works, examine this short program:

```
// An example of a friend function.
#include <iostream>
using namespace std;

class myclass
{
    int n, d;
public:
    myclass(int i, int j) { n = i; d = j; }
    // declare a friend of myclass
    friend int isfactor(myclass ob);
};

/*
   Here is friend function definition. It returns true
   if d is a factor of n. Notice that the keyword friend is not
      used in the definition of isfactor()
*/
int isfactor(myclass ob)
{
    if(!(ob.n % ob.d))
        return 1;
    else
        return 0;
}

int main()
{
    myclass ob1(10, 2), ob2(13, 3);

    if(isfactor(ob1))
        cout << "2 is a factor of 10\n";
    else
        cout << "2 is not a factor of 10\n";

    if(isfactor(ob2))
        cout << "3 is a factor of 13\n";
    else
        cout << "3 is not a factor of 13\n";

    return 0;
}
```

In this example, **myclass** declares its constructor function and the friend **isfactor()** inside its class declaration. Because **isfactor()** is a friend of **myclass**, **isfactor()** has access to its private members. This is why, within **isfactor()**, it is possible to directly refer to **ob.n** and **ob.d**.
It is important to understand that a friend function is not a member of the class for which it is a friend. Thus, it is not possible to call a friend function by using an object name and a class member access operator (a dot or an arrow). For example, given the preceding example, this

statement is wrong:

```
ob1.isfactor(); // wrong; isfactor() is not a member function
```

Instead, friends are called just like regular functions.
Although a friend function has knowledge of the private elements of the class for which it is a friend, it can only access them through an object of the class. That is, unlike a member function of **myclass**, which can refer to **n** or **d** directly, a friend can access these variables only in conjunction with an object that is declared within or passed to the friend function.
Note: *The preceding paragraph brings up an important side issue. When a member function refers to a private element, it does not directly because a member function is executed only in conjunction with an object of that class. Thus, when a member function refers to a private element, the compiler knows which object that private element belongs to by the object that is linked to the function when that member function is called. However a friend function is not linked to any object. It simply is granted access to the private elements of a class. Thus, inside the friend function, it is meaningless to refer to a private member without reference to a specific object.*
Because friends are not members of a class, they will typically be passed one or more objects of the class for which they are friends. This is the case with **isfactor()**. It is passed an object of **myclass**, called **ob**. However, because **isfactor()** is a friend of **myclass**, it can access **ob**'s private elements. If **isfactor()** had not been made a friend of **myclass**, it would not be able to access **ob.d** or **ob.n** since **n** and **d** are private members of **myclass**.
Remember: *A friend function is not a member and cannot be qualified by an object name. It must be called just like a normal function.*
A friend function is not inherited. That is, when a base class includes a friend function, that friend function is not a friend of a derived class.
One other important point about friend functions is that a friend function can be friends with more than one class.

EXAMPLES

1. One common (and good) use of a friend function occurs when two different types of classes have some quantity in common that needs to be compared. For example, consider the following program, which creates a class called **car** and a class called **truck**, each containing, as a private variable, the speed of the vehicle it represents:

```
#include <iostream>
using namespace std;

class truck; // a forward declaration

class car
{
    int passengers;
    int speed;
public:
    car(int p, int s) { passengers = p; speed = s; }
    friend int sp_greater(car c, truck t);
};

class truck
{
    int weight;
```

```
    int speed;
public:
    truck(int w, int s) { weight = w; speed = s; }
    friend int sp_greater(car c, truck t);
};

/*
   Return positive if car speed faster than truck.
   Return 0 if speeds are the same.
   Return negative if truck speed faster than car.
*/
int sp_greater(car c, truck t)
{
    return c.speed - t.speed;
}

int main()
{
    int t;
    car c1(6, 55), c2(2, 120);
    truck t1(10000, 55), t2(20000, 72);

    cout << "Comparing c1 and t1:\n";
    t = sp_greater(c1, t1);
    if(t<0)
        cout << "Truck is faster.\n";
    else if(t==0)
        cout << "Car and truck speed is the same.\n";
    else
        cout << "Car is faster.\n";

    cout << "Comparing c2 and t2:\n";
    t = sp_greater(c2, t2);
    if(t<0)
        cout << "Truck is faster.\n";
    else if(t==0)
        cout << "Car and truck speed is the same.\n";
    else
        cout << "Car is faster.\n";

    return 0;
}
```

This program contains the function **sp_greater()**, which is a friend function of both the **car** and **truck** classes. (As stated, a function can be a friend of two or more classes.) This function returns positive if the **car** object is going faster than the **truck** object, 0 if their speeds are the same, and negative if the **truck** is going faster.

This program illustrates one important C++ syntax element the *forward declaration* (also called *forward reference*. Because **sp_greater()** takes parameters of both the **car** and the **truck** classes, it is logically impossible to declare both before including **sp_greater()** in either. Therefore, there needs to be some way to tell the compiler about a class name

without actually declaring it. This is called a forward declaration. In C++, to tell the compiler that an identifier is the name of a class, use a line like this before the class name is first used:

```
class class_name;
```

For example, in the preceding program, the forward declaration is:

```
class truck;
```

Now **truck** can be used in the friend declaration of **sp_greater()** without generating a compile-time error.

2. A function can be a member of one class and a friend of another. For example, here is the preceding example rewritten so that **sp_greater** is a member of **car** and a friend of **truck**:

```
#include <iostream>
using namespace std;

class truck; // a forward declaration

class car
{
    int passengers;
    int speed;
public:
    car(int p, int s) { passengers = p; speed = s; }
    int sp_greater(truck t);
};

class truck
{
    int weight;
    int speed;
public:
    truck(int w, int s) { weight = w; speed = s; }

    // note new use of the scope resolution operator
    friend int car::sp_greater(truck t);
};

/*
   Return positive if car speed faster than truck.
   Return 0 if speeds are the same.
   Return negative if truck speed faster than car.
*/
int car::sp_greater(truck t)
{
    /*
       Since sp_greater() is member of car, only a
       truck object must be passed to it.
    */
    return speed - t.speed;
```

```
}

int main()
{
    int t;
    car c1(6, 55), c2(2, 120);
    truck t1(10000, 55), t2(20000, 72);

    cout << "Comparing c1 and t1:\n";
    t = c1.sp_greater(t1); // evoke as member function of car
    if(t<0)
        cout << "Truck is faster.\n";
    else if(t==0)
        cout << "Car and truck speed is the same.\n";
    else
        cout << "Car is faster.\n";

    cout << "Comparing c2 and t2:\n";
    t = c2.sp_greater(t2); // evoke as member function of car
    if(t<0)
        cout << "Truck is faster.\n";
    else if(t==0)
        cout << "Car and truck speed is the same.\n";
    else
        cout << "Car is faster.\n";

    return 0;
}
```

Notice the new use of the scope resolution operator as it occurs in the friend declaration within the **truck** class declaration. In this case, it is used to tell the compiler that the function **sp_greater()** is a member of the **car** class.

One easy way to remember how to use the scope resolution operator is that the class name followed by the scope resolution operator followed by the member name fully specifies a class member.

In fact, when referring to a member of a class, it is never wrong to fully specify its name. However, when an object is used to call a member function or access a member variable, the full name is redundant, and seldom used. For example,

```
t = c1.sp_greater(t1);
```

can be written using the (redundant) scope resolution operator and the class name **car** like this:

```
t = c1.car::sp_greater(t1);
```

However, since **c1** is an object of type **car**, the compiler already knows that **sp_greater()** is a member of the **car** class, making the full class specification unnecessary.

EXERCISE

1. Imagine a situation in which two classes, called **pr1** and **pr2**, shown here, share one printer. Further, imagine that other parts of your program need to know when the printer

is in use by an object of either of these two classes. Create a function called **inuse()** that returns **true** when the printer is being used by either and **false** otherwise. Make this function a friend of both **pr1** and **pr2**.

```
class pr1
{
    int printing;
    // ...
public:
    pr1() { printing = 0; }
    void set_print(int status) { printing = status; }
    // ...
};

class pr2
{
    int printing;
    // ...
public:
    void set_print(int status) { printing = status; }
    // ...
};
```

SKILLS CHECK

Mastery Skills Check

Before proceeding, you should be able to answer the following questions and perform the exercises.

1. What single prerequisite must be met in order for one object to be assigned to another?
2. Given this class fragment,

```
class samp
{
    double *p;
public:
    samp(double d)
    {
        p = (double *) malloc(sizeof(double));
        if(!p)
            exit(1); // allocation error
        *p = d;
    }
    ~samp() { free(p); }
    // ..
};

// ...
samp ob1(123.09), ob2(0.0);
// ...
ob2 = ob1;
```

what problem is caused by the assignment of **ob1** to **ob2**?

3. Given this class,

```
class planet
{
    int moons;
    double dist_from_sun; // in miles
    double diameter;
    double mass;
public:
    // ....
    double get_miles() { return dist_from_sun; }
};
```

create a function called **light()** that takes as an argument an object of type **planet** and returns the number of seconds that it takes light from the sun to reach the planet. (Assume that light travels at 186,000 miles per second and that **dist_from_sun** is specified in miles.)

4. Can the address of an object be passed to a function as an argument?

5. Using the **stack** class, write a function called **loadstack()** that returns a stack that is already loaded with the letters of the alphabet(a-z). Assign this stack to another object in the calling routine and prove that it contains the alphabet. Be sure to change the stack size so it is large enough to hold the alphabet.

6. Explain why you must be careful when passing objects to a function or returning objects from a function.

7. What is a friend function?

Cumulative Skills Check

This section checks how well you have integrated material in this chapter with that from the preceding chapters.

1. Functions can be overloaded as long as the number or type of their parameters differs. Overload **loadstack()** from Exercise 5 of the Mastery Skills Check so that it takes an integer, called **upper**, as a parameter. In the overloaded version, if **upper** is 1, load the stack with the uppercase alphabet. Otherwise, load it with the lowercase alphabet.

2. Using the **strtype** class shown in Section 3.1, Example 3, add a friend function that takes as an argument a pointer to an object of type **strtype** and returns a pointer to the string pointed to by that object. (That is, have the function return **p**.) Call this function **get_string()**.

3. Experiment: When an object of a derived class is assigned to another object of the same derived class, is the data associated with the base class also copied? To find out, use the following two classes and write a program that demonstrates what happens.

```
class base
{
    int a;
public:
    void load_a(int n) { a = n; }
```

```
    int get_a() { return a; }
};

class derived : public base
{
    int b;
public:
    void load_b(int n) { b = n; }
    int get_b() { return b; }
};
```